

Cost Optimization in Agent Systems

Token economics, caching, model routing, and the engineering practices that cut agent costs by 60 to 80 percent without sacrificing output quality.



Rajesh Mane
Senior AI Engineer



Why Agent Costs Compound Fast

 Individual API calls cost fractions of a cent but agents execute dozens of steps per task. Costs compound with every loop iteration, tool call, and context expansion.

 Output tokens cost 3x to 10x more than input tokens. This pricing asymmetry makes verbose responses and unnecessary chain of thought reasoning the most expensive habits.

 Context grows with every turn as conversation history, tool results, and system prompts accumulate. You pay for the full context on every subsequent call.

 Multi agent systems consume 4x to 15x more tokens than single agent workflows if not optimized. Each agent carries its own system prompt, context, and tool schemas.

THE REAL PROBLEM

The problem is not that LLMs are expensive per token. The problem is how agents use tokens: long context windows, repeated system prompts, verbose outputs, and unbounded loops.



Three Cost Categories to Track



INPUT TOKENS: CONTEXT YOU SEND

System prompts, conversation history, tool results, and retrieved documents. These repeat and grow every turn. Prompt caching targets this category directly with up to 90 percent savings.



OUTPUT TOKENS: THE EXPENSIVE SIDE

The text the model generates. Typically 3x to 10x more expensive than input. Structured output (JSON mode) and concise response instructions reduce this significantly.



INFRASTRUCTURE: THE HIDDEN COSTS

Vector databases, embedding generation, file storage, and compute for tool execution. Often overlooked in token cost analyses but scales linearly with data volume and query frequency.



Prompt Caching: The Biggest Win

UP TO 90 PERCENT SAVINGS ON CACHED TOKENS

1 System prompts repeat identically every request

Persona definitions, tool schemas, and instructions are sent with every call. Prompt caching lets the provider remember this prefix and charge a discounted read rate instead of full processing.

2 Structure prompts with static content first

The cache matches the prefix of your prompt. Fixed instructions must come before dynamic content like user messages and tool results so the cache hit rate stays high.

3 Savings scale with prompt length and call volume

A 10,000 token system prompt called thousands of times per day saves 90 percent on those cached tokens. For agents with long prompts, this is the single highest impact optimization.



START HERE BEFORE ANYTHING ELSE

Prompt caching requires minimal code changes, works with existing prompts, and delivers the largest savings per unit of effort. Enable it before optimizing anything else.



Model Routing: Right Model per Step

ROUTE SIMPLE TASKS TO CHEAPER MODELS

1 Not every step needs a frontier model

Simple classification, extraction, and formatting run on models that cost 10x to 30x less with no measurable quality loss. Reserve frontier models for tasks requiring deep reasoning.

2 Audit your call log to find routing candidates

Identify which agent steps need reasoning and which are mechanical. Most agents have 4 to 6 steps where a smaller model performs identically to a premium one.

3 LLM gateways handle routing automatically

LiteLLM, Portkey, and OpenRouter support routing rules, fallback chains, and per model cost tracking. The router adds milliseconds of latency but saves dollars per day.



THE 75 PERCENT CASE STUDY

Routing 4 of 7 agent steps to a mid tier model and 2 to a small model reduced per task cost from \$1.40 to \$0.34. Same output quality, 75 percent less spend.



Spent

MINIMUM CONTEXT FOR MAXIMUM REASONING

Every token in the context window costs money. Reducing what goes in reduces what you pay without sacrificing reasoning quality.

CONTEXT WINDOWING

Trim older conversation turns
beyond a sliding window.

Summarize instead of carrying full history. Keep only the turns the agent needs for the current decision.

RAG OVER STUFFING

Retrieve only relevant chunks
instead of stuffing entire
documents into the prompt.

Targeted retrieval cuts input tokens by an order of magnitude on document heavy tasks.

 MEASURE QUALITY ALONGSIDE COST

Aggressive context compression can degrade reasoning. Always test accuracy when tuning window size and retrieval depth. The cheapest prompt that gives wrong answers saves nothing.



Semantic Caching: Reuse Answers

SAME INTENT, NO RECOMPUTATION

1 Prompt caching matches prefixes, semantic caching matches intent

When users ask the same question in different words, semantic caching returns a validated answer without calling the LLM at all.

2 Embed queries and check similarity against cached pairs

If the incoming query is similar enough to a previous one above a confidence threshold, serve the cached response directly. The model is never invoked.

3 Apply selectively to low risk, high frequency queries

Support agents, FAQ systems, and documentation assistants benefit most. Sensitive, personalized, or time dependent queries should always hit the model fresh.



DRAMATIC SAVINGS ON REPETITIVE TRAFFIC

Semantic caching can reduce API calls dramatically on workloads where users repeatedly ask the same questions in different phrasing. The savings grow with traffic volume.



Cost Strategies: Side by Side

Strategy	Savings	Effort	Best For
Prompt caching	Up to 90%	Low	Long system prompts
Model routing	40-60%	Medium	Multi step agents
Context windowing	30-50%	Medium	Long conversations
Semantic caching	Varies	Medium	Repetitive queries
Structured output	20-40%	Low	Verbose responses
Batch APIs	50%	Low	Background tasks



START WITH THE TOP TWO

Prompt caching and model routing together deliver 40 to 60 percent savings with the lowest implementation effort. Add the others incrementally as volume grows.



COST GOVERNANCE AT EVERY LEVEL

Aggregate dashboards hide the fact that one misbehaving agent may consume 80 percent of your total spend. Granular tracking exposes the real cost drivers.



TREAT TOKENS LIKE CLOUD COMPUTE

Cost governance is not optional at scale. Apply the same FinOps discipline to token budgets that you apply to cloud compute. Set limits before production, not after the first surprise bill.



Structured Output Cuts Output Tokens

THE MOST EXPENSIVE TOKEN IS THE ONE YOU GENERATE

1 JSON mode returns only the fields you need

Instead of verbose free text explanations, structured schemas force the model to return exact data. A boolean and a confidence score replace three paragraphs of justification.

2 Suppress chain of thought in production

When intermediate reasoning is not needed downstream, skip it. The model reasons internally but returns only the final answer, cutting output tokens dramatically.

3 Define schemas that match the next step's input

Each tool call should receive exactly the data it needs. Over specifying response formats wastes output tokens that the consuming function will ignore anyway.



THE 4x TO 8x MULTIPLIER

The output to input cost ratio is typically 4x to 5x and reaches 8x on premium reasoning models. Every token cut from output saves more than every token cut from input.



Multi Agent Cost Traps

WHEN MORE AGENTS MEANS MORE WASTE

1 Each agent carries its own context overhead

System prompts, tool schemas, and conversation history multiply with every additional agent. Token costs scale linearly with the number of agents in the pipeline.

2 Communication between agents adds token overhead

The coordinator summarizes, the worker parses, the result flows back. Each handoff is a round trip of tokens that a single agent workflow would not need.

3 Use multi agent only for truly independent tasks

If work is sequential and fits one agent's context, adding agents adds cost without adding capability. Justify every additional agent with a measurable output improvement.

⚠️ 4x TO 15x MORE TOKENS

Multi agent systems consume 4x to 15x more tokens than optimized single agent workflows. Measure whether the capability gain justifies the cost before adding agents.



Your Cost Optimization Playbook

FOUR STEPS IN PRIORITY ORDER

1

Enable Prompt Caching

Minimal code changes, up to 90 percent savings on cached input tokens. This is the highest return on effort optimization available for any agent system.

2

Route Models by Complexity

Audit your call log, identify mechanical steps, route them to cheaper models. Keep frontier models only for tasks that require deep reasoning.

3

Set Token Budgets Everywhere

Per request, per task, per day.
Alert at thresholds. A single runaway agent loop can consume your entire daily budget in minutes.

4

Measure Cost per Outcome

Track cost per resolved ticket, per qualified lead, per completed task. Aggregate token counts hide which workflows are efficient and which waste budget.



Cost Mistakes to Avoid

HABITS THAT DRAIN YOUR TOKEN BUDGET

-  Running every step on a frontier model when 80 percent of steps are simple classification or extraction that a smaller model handles equally well.
-  Skipping prompt caching because the system prompt seems short. Even moderate prompts accumulate significant cost when called thousands of times per day.
-  Carrying full conversation history without windowing. Context grows linearly with turns, and you pay for the full window on every subsequent call.
-  Ignoring output token costs. They are 3x to 10x more expensive than input. Verbose responses and chain of thought reasoning are the costliest habits to leave unchecked.
-  Launching multi agent systems without checking if a single optimized agent produces the same result at a fraction of the cost.



A Cost Engineering Reality Check

“

The cheapest token is the one you never send.

The second cheapest is the one you cache.

Everything else is optimization at the margin.

— Rajesh Mane, Senior AI Engineer



Key Takeaways



Prompt caching is the biggest win

Up to 90 percent savings on cached input tokens with minimal code changes. Start here before any other optimization.



Model routing saves 40 to 60 percent

Route simple steps to cheaper models. Keep frontier models for deep reasoning. The router pays for itself in hours.



Output tokens are the hidden cost

3x to 10x more expensive than input. Structured output, JSON mode, and suppressed chain of thought cut this fastest.



Measure cost per outcome

Aggregate dashboards hide waste. Track cost per resolved ticket, per qualified lead, per completed workflow to find real targets.



Follow Rajesh Mane for daily AI insights

I share practical AI and ML content on topics like machine learning, deep learning, GenAI, Agentic AI, RAG, LLMs and MLOps daily.



Like



Comment



Repost



Rajesh Mane | Senior AI Engineer

